

GrovLock: Diseño e Implementación de un Algoritmo Híbrido de Cifrado para Comunicaciones Seguras

Alegre Castilla, Sthefany Alexandra*
Pareja Abarca, María Julia*

Espino Veas, Karla Daniela*

Panclas Aliaga, María Claudia*

*Universidad Católica de Santa María

Borrador / Preprint. Este documento es un borrador en revisión activa. El contenido, las figuras y las conclusiones pueden cambiar antes de una eventual versión final. Comentarios y correcciones son bienvenidos a través de GitHub Issues. xd

Abstract—Este trabajo presenta GrovLock, un esquema criptográfico híbrido diseñado para proteger comunicaciones telemáticas mediante la integración de mecanismos de intercambio de claves, derivación criptográfica, difusión de datos y cifrado autenticado. La arquitectura propuesta combina X25519 (ECDH) para la generación segura de secretos compartidos, una función de derivación de claves basada en SHA-256 para obtener claves de sesión y AES-256-GCM para garantizar confidencialidad, integridad y autenticidad. Como principal aporte, GrovLock incorpora una capa de difusión reversible inspirada en el principio matemático del operador de difusión de Grover, adaptado a un entorno clásico. Esta capa transforma bloques de 128 bits antes del cifrado, incrementando la dispersión de los datos y reduciendo patrones identificables. El algoritmo fue implementado en Python y evaluado en una arquitectura cliente-servidor TCP/IP y en un bot de Telegram. Los resultados demostraron un intercambio seguro de claves, transmisión confiable de mensajes y correcta recuperación de la información, validando la viabilidad de la propuesta para comunicaciones telemáticas seguras.

Index Terms—criptografía híbrida, ECDH, X25519, AES-256-GCM, operador de difusión de Grover, cifrado autenticado, función de derivación de clave.

I. INTRODUCCIÓN

La transformación digital ha incrementado significativamente el intercambio de información a través de aplicaciones de mensajería, sistemas cliente-servidor y servicios distribuidos conectados mediante redes públicas, y como consecuencia, la protección de la confidencialidad, integridad y autenticidad de los datos transmitidos se ha convertido en un requisito fundamental para garantizar la seguridad de las comunicaciones telemáticas. Aunque los algoritmos criptográficos modernos ofrecen altos niveles de protección, la aparición de nuevas amenazas y el avance de la computación cuántica han impulsado la investigación de mecanismos innovadores que permitan fortalecer los esquemas de seguridad existentes.

En los últimos años, diversos estudios han propuesto arquitecturas criptográficas híbridas orientadas a fortalecer la seguridad de las comunicaciones digitales mediante la integración de mecanismos de intercambio de claves, funciones

de derivación criptográfica y algoritmos de cifrado autenticado. Según Luc et al. [1], la incorporación de técnicas de criptografía post-cuántica en aplicaciones de mensajería permite mejorar la protección de la información frente a amenazas emergentes derivadas del avance de la computación cuántica, manteniendo al mismo tiempo niveles adecuados de rendimiento.

De manera parecida, Fauziyah, Wang y Tabassum [2] señalan que los protocolos multicapa que combinan diferentes mecanismos criptográficos ofrecen una mayor resistencia frente a ataques de interceptación, manipulación y acceso no autorizado, debido a que cada componente contribuye a proteger una fase específica del proceso de comunicación. Es así como estos enfoques han impulsado el desarrollo de modelos híbridos capaces de aprovechar las ventajas de algoritmos asimétricos para el intercambio seguro de claves y de algoritmos simétricos para el cifrado eficiente de datos, constituyéndose como una alternativa prometedora para garantizar la seguridad de las comunicaciones telemáticas modernas.

Asimismo, el cifrado autenticado mediante AES-GCM o Advanced Encryption Standard, Galois/Counter Mode, continúa siendo una de las tecnologías más utilizadas para la protección de comunicaciones digitales debido a su capacidad para proporcionar simultáneamente confidencialidad, integridad y autenticación de los datos transmitidos. Según Mandal et al. [3], los esquemas basados en AES-GCM ofrecen una combinación eficiente entre seguridad criptográfica y rendimiento computacional, razón por la cual han sido adoptados ampliamente en protocolos modernos de comunicación segura, servicios en la nube, aplicaciones de mensajería y sistemas de transmisión de datos. A diferencia de los mecanismos de cifrado convencionales que únicamente protegen el contenido de la información, AES-GCM incorpora etiquetas de autenticación que permiten detectar cualquier modificación no autorizada realizada durante la transmisión, reduciendo significativamente el riesgo de ataques de manipulación o alteración de mensajes. Por otro lado, su diseño basado en operaciones paralelizables favorece una implementación eficiente tanto en software como en hardware, permitiendo mantener altos niveles de rendimiento incluso en entornos con grandes volúmenes de tráfico; así, estas características han consolidado a AES-GCM como uno de los estándares más

confiables para garantizar la seguridad de las comunicaciones telemáticas modernas.

Paralelamente, el avance de la computación cuántica ha impulsado el estudio de algoritmos capaces de transformar significativamente los paradigmas tradicionales de procesamiento y seguridad de la información. Entre ellos destaca el algoritmo de búsqueda de Grover, propuesto originalmente para acelerar la localización de elementos dentro de bases de datos no estructuradas mediante un proceso iterativo de amplificación de amplitudes cuánticas. Según Yoon et al. [4], las propiedades matemáticas derivadas de este algoritmo han despertado un creciente interés en la comunidad científica debido a su potencial aplicación en nuevas propuestas de seguridad y procesamiento de información; no obstante, es importante señalar que el algoritmo de Grover no constituye un mecanismo de cifrado ni fue diseñado para proteger datos, sino para optimizar procesos de búsqueda en sistemas cuánticos.

En este contexto, investigaciones recientes como la de Al-Bayaty y Perkowski [5] han analizado las características del operador de difusión de Grover, destacando su capacidad para realizar transformaciones reversibles basadas en reflexiones respecto a la media de un conjunto de valores. Esta propiedad permite modificar la distribución de los datos preservando la información necesaria para recuperar el estado original, lo que abre la posibilidad de adaptar dicho principio matemático a nuevas arquitecturas criptográficas. En consecuencia, la utilización de mecanismos inspirados en el operador de difusión de Grover representa una alternativa innovadora para incrementar la transformación y dispersión de los datos antes de aplicar procesos de cifrado convencionales, contribuyendo potencialmente al fortalecimiento de la seguridad en las comunicaciones telemáticas.

En este contexto, el presente trabajo propone GrovLock, un esquema criptográfico híbrido diseñado para proteger comunicaciones telemáticas mediante la integración de X25519 (ECDH) para el intercambio seguro de claves, una función de derivación de claves (KDF) y AES-256-GCM para el cifrado autenticado de los datos. Como principal aporte, GrovLock incorpora una capa de difusión reversible inspirada en el operador de difusión de Grover, adaptada a un entorno clásico y aplicada sobre bloques de 128 bits antes del cifrado. De esta manera, se incrementa la transformación y dispersión de los datos, fortaleciendo la protección de la información transmitida en entornos cliente-servidor y sistemas de mensajería.

II. METODOLOGÍA

A. Diseño del algoritmo GrovLock

La metodología propuesta consiste en el diseño e implementación de GrovLock, como lo nombramos, un esquema criptográfico híbrido orientado a la protección de comunicaciones telemáticas. La arquitectura integra mecanismos de criptografía asimétrica y simétrica con una capa de difusión inspirada en el operador de difusión de Grover.

El objetivo es incrementar la transformación de los datos antes del proceso de cifrado, añadiendo una etapa adicional de difusión sin afectar la capacidad de recuperación de la

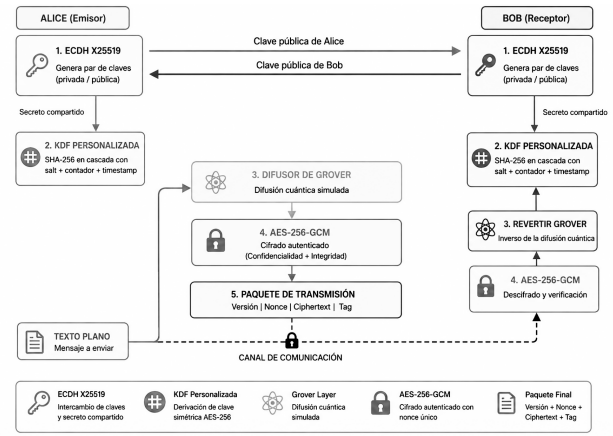


Fig. 1. Arquitectura propuesta

información original. El algoritmo está compuesto por cuatro etapas principales: intercambio seguro de claves, derivación de claves de sesión, difusión de datos y cifrado autenticado. La Fig. 1 muestra la arquitectura general propuesta.

1) *Intercambio de claves*: La primera etapa de GrovLock consiste en el establecimiento de un secreto compartido mediante el protocolo X25519, una implementación del mecanismo Elliptic Curve Diffie-Hellman (ECDH) basada en la curva elíptica Curve25519. Según Nguyen et al. [6], este protocolo permite que dos entidades que se comunican a través de una red insegura generen de forma independiente una misma clave secreta sin necesidad de transmitirla directamente.

Para ello, tanto el emisor como el receptor generan un par de claves compuesto por una clave privada y una clave pública; posteriormente, intercambian únicamente sus claves públicas a través del canal de comunicación. Utilizando su propia clave privada y la clave pública de la otra parte, cada participante ejecuta la operación ECDH para calcular un secreto compartido idéntico. Debido a las propiedades matemáticas de la criptografía de curva elíptica, ambos extremos obtienen el mismo resultado sin exponer sus claves privadas ni la clave de sesión derivada.

```

37 #
38 # FASE 1: CLAVES ECDH (X25519 / Curva25519)
39 #
40
41 class ParDeClaves:
42     def __init__(self):
43         self.clave_privada = X25519PrivateKey.generate()
44         self.clave_publica = self.clave_privada.public_key()
45
46     def exportar_publica_b64(self) -> str:
47         raw = self.clave_publica.public_bytes(Encoding.Raw, PublicFormat.Raw)
48         return base64.b64encode(raw).decode()
49
50     @staticmethod
51     def importar_publica(b64: str) -> X25519PublicKey:
52         raw = base64.b64decode(b64)
53         return X25519PublicKey.from_public_bytes(raw)

```

Fig. 2. Código para claves X25519

La operación exchange() permite que ambos participantes obtengan el mismo secreto compartido sin transmitirlo directamente por la red.

La utilización de X25519 aporta ventajas significativas en términos de seguridad y rendimiento, ya que proporciona

```
secreto_ecdh = self._par.clave_privada.exchange(pub_remota)
```

Fig. 3. Operación de exchange

resistencia frente a ataques conocidos contra el intercambio de claves, reduce el costo computacional en comparación con otros esquemas asimétricos tradicionales y es ampliamente utilizado en protocolos modernos de comunicación segura. En GrovLock, el secreto compartido generado mediante X25519 constituye la base para la derivación de la clave de sesión utilizada posteriormente por la capa de difusión y el proceso de cifrado AES-256-GCM.

2) *Derivación de claves:* Una vez establecido el secreto compartido mediante X25519, este no se utiliza directamente como clave de cifrado. En su lugar, se procesa mediante una función de derivación de claves (KDF) basada en SHA-256 con el propósito de generar una clave de sesión criptográficamente robusta y adecuada para las etapas posteriores del algoritmo. Las funciones KDF permiten transformar material criptográfico inicial en claves pseudoaleatorias de alta entropía, reduciendo posibles sesgos presentes en la entrada y garantizando propiedades adecuadas para su uso en mecanismos de cifrado y autenticación.

Según Chuah, Harun y Hamid [7], las KDF constituyen un componente fundamental de los protocolos de seguridad modernos, ya que permiten generar una o más claves criptográficas a partir de información compartida de longitud arbitraria. Asimismo, Bhati et al. [8] destacan que HKDF es actualmente uno de los mecanismos de derivación más utilizados en protocolos de intercambio de claves y sistemas de mensajería segura debido a su enfoque extract-then-expand, el cual produce claves uniformemente distribuidas y resistentes a ataques criptográficos.

En GrovLock, la KDF genera una clave de sesión de 256 bits, utilizada posteriormente tanto para parametrizar la capa de difusión inspirada en Grover como para el proceso de cifrado autenticado mediante AES-256-GCM.

```
56 # -----
57 # FASE 2: KDF PERSONALIZADA
58 # -----
59
60 def kdf_propio(secreto: bytes, contador: int) -> bytes:
61     """
62     SHA256( secreto || SALT || SHA256( contador || timestamp ) )
63     Produce 32 bytes + clave AES-256.
64     """
65     ts_bytes = struct.pack(">Q", int(time.time()))
66     cnt_bytes = struct.pack(">Q", contador)
67     inner = hashlib.sha256(cnt_bytes + ts_bytes).digest()
68     return hashlib.sha256(secreto + SALT_PERSONALIZADO + inner).digest()
69
70
71 def generar_nonce(contador: int) -> bytes:
72     """SHA256( microsegundos || contador || random )[:12]"""
73     ts = struct.pack(">Q", int(time.time()) * 1_000_000)
74     cnt = struct.pack(">Q", contador)
75     rnd = os.urandom(4)
76     return hashlib.sha256(ts + cnt + rnd).digest()[:12]
```

Fig. 4. Código para KDF

3) *Capa de difusión:* La principal contribución de GrovLock es la incorporación de una capa de difusión reversible inspirada en el operador de difusión de Grover; es importante destacar que la propuesta no implementa el algoritmo

cuántico de búsqueda de Grover ni requiere infraestructura de computación cuántica. En su lugar, adopta únicamente el principio matemático de reflexión respecto a la media y lo adapta a un entorno clásico de procesamiento de datos con el objetivo de incrementar la dispersión de la información antes del proceso de cifrado.

La difusión se aplica sobre bloques de 128 bits (16 bytes) y para cada bloque, se calcula inicialmente la media de los valores que lo componen y posteriormente cada byte es transformado mediante una operación de reflexión respecto a dicha media. La transformación utilizada se define como:

$$D(v[i]) = (2\mu - v[i]) \bmod 256 \quad (1)$$

donde $v[i]$ representa el valor original del byte i , μ corresponde a la media de todos los bytes del bloque y $D(v[i])$ representa el valor obtenido después de aplicar la difusión; de este modo, esta operación genera una redistribución de los valores dentro del bloque, incrementando la dispersión de los datos y reduciendo la presencia de patrones identificables antes de la etapa de cifrado.

Según Al-Bayaty y Perkowski [5], los principios matemáticos asociados al operador de difusión de Grover pueden adaptarse para realizar transformaciones reversibles sobre conjuntos de datos; basándose en este concepto, GrovLock implementa una variante clásica de dicha transformación como mecanismo adicional de mezcla previo al cifrado autenticado. De la misma forma, el número de rondas de difusión es derivado dinámicamente a partir de la clave de sesión generada mediante X25519 y la función KDF. Esto introduce variabilidad entre diferentes sesiones de comunicación, de modo que mensajes similares pueden experimentar procesos de transformación distintos; finalmente, debido a que la operación es reversible, el receptor puede recuperar exactamente los datos originales antes del descifrado, preservando la integridad de la información transmitida.

En sí, en lugar de ejecutar el algoritmo cuántico completo, se adaptó matemáticamente el operador de reflexión respecto al promedio para trabajar sobre bloques de bytes.

4) *Cifrado autenticado:* Una vez aplicada la capa de difusión, los datos son cifrados mediante AES-256-GCM, utilizando la clave de sesión derivada durante el intercambio de claves. Este algoritmo fue seleccionado debido a que proporciona simultáneamente confidencialidad, integridad y autenticidad de la información transmitida. Además, incorpora mecanismos de verificación que permiten detectar cualquier modificación no autorizada de los datos durante la transmisión. De esta manera, GrovLock garantiza que únicamente los usuarios que comparten la clave de sesión puedan acceder al contenido original de los mensajes.

Según Mandal et al. [3], los esquemas AEAD o Authenticated Encryption with Associated Data basados en AES-GCM continúan siendo una de las alternativas más robustas y ampliamente adoptadas para la protección de comunicaciones digitales debido a su equilibrio entre seguridad criptográfica,

```

39 def _difusor_grover_bloque(bloque: List[int], rondas: int) -> List[int]:
40     """
41     Aplica el difusor de Grover a un bloque de bytes.
42
43     Para garantizar invertibilidad exacta sin pérdidas de redondeo,
44     calculamos la suma (en vez del promedio) y operamos con enteros:
45
46     suma = Σ v[i]
47     w[i] = (2*suma//N - v[i]) mod 256
48
49     Esto es equivalente a D(v)[i] = (2μ - v[i]) mod 256 pero con
50     aritmética entera exacta cuando N divide a 2*suma.
51     """
52     v = list(bloque)
53     n = len(v)
54
55     for _ in range(rondas):
56         S = sum(v) # Suma total (entero exacto)
57         # 2*μ·N = 2·S/N·N = 2S, pero necesitamos 2*μ = 2S/N
58         # Usamos: w[i] = (2*S - N*v[i]) // N si N | (2S - N*v[i])
59         # Más simple: trabajamos con escala N para mantener enteros
60         doble_media_n = 2 * S # = 2*μ·N (sin dividir)
61         v = [(doble_media_n - n * vi) % (256 * n) // n for vi in v]
62         # Simplificado: (2S/N - vi) mod 256 con aritmética entera
63         v = [(2 * S // n - vi) % 256 for vi in v]
64
65     return v

```

Fig. 5. Código para el difusor de bloque

```

83 def cifrar(clave: bytes, mensaje: str, contador: int) -> str:
84
85     # Fase 3: capa Grover
86     datos_planos = mensaje.encode("utf-8")
87     datos_grover = aplicar_grover(datos_planos, clave)
88
89     # Fase 4: AES-256-GCM
90     nonce = generar_nonce(contador)
91     aesgcm = AESGCM(clave)
92     cifrado = aesgcm.encrypt(nonce, datos_grover, associated_data=None)
93
94     # Empaquetar: versión(1) + nonce(12) + ciphertext+tag
95     paquete = VERSION_PROTOCOLO + nonce + cifrado
96     return base64.b64encode(paquete).decode()

```

Fig. 6. Función para cifrar

eficiencia computacional y capacidad de detección de modificaciones no autorizadas. Por esta razón, GrovLock utiliza AES-256-GCM como mecanismo final de protección, complementando la difusión previa de los datos con un esquema de cifrado autenticado reconocido internacionalmente.

B. Implementación experimental

La implementación de GrovLock se desarrolló utilizando el lenguaje de programación Python debido a su amplia disponibilidad de bibliotecas criptográficas y de comunicación en red. Con el propósito de validar el funcionamiento del algoritmo en escenarios de comunicación reales, se realizaron pruebas en entornos distintos.

```

110 def descifrar(clave: bytes, b64_texto: str) -> str:
111
112     paquete = base64.b64decode(b64_texto)
113
114     version = paquete[:1]
115     nonce = paquete[1:13]
116     cifrado = paquete[13:]
117
118     # Fase 4 inversa: AES-GCM
119     aesgcm = AESGCM(clave)
120     datos_grover = aesgcm.decrypt(nonce, cifrado, associated_data=None)
121
122     # Fase 3 inversa: revertir Grover
123     datos_planos = revertir_grover(datos_grover, clave)
124     return datos_planos.decode("utf-8")

```

Fig. 7. Función para descifrar

C. Aplicación telemática o bot de Telegram

El bot implementa una capa de cifrado end-to-end adicional sobre Telegram. Telegram ya cifra sus propias comunicaciones con el protocolo MTProto, pero con nuestro sistema añadimos una segunda capa: los mensajes que viajan por los servidores de Telegram son texto cifrado Base64 completamente ilegible para Telegram o cualquier intermediario.

1) Componentes del sistema:

- **bot.py**: el servidor del bot que corre en la laptop del desarrollador. Registra los handlers para cada comando y gestiona la entrega de mensajes entre usuarios usando la API de Telegram.
- **session_manager.py**: gestiona en memoria el estado de todas las sesiones activas. Mantiene los pares de claves, solicitudes pendientes de conexión, claves de sesión derivadas y contadores de mensaje.
- **hybrid_cipher_v2.py + grover_layer.py**: el núcleo criptográfico, invocado por el manager en cada operación de cifrado o descifrado.

TABLE I
COMANDOS DEL BOT

Comando	Descripción
/start	Bienvenida e instrucciones del bot
/miid	Muestra tu Telegram User ID (número necesario para conectarse)
/nuevasesion	Genera par de claves X25519 e inicia el proceso de handshake
/conectar	/conectar ¡id _i ! — Solicita conexión cifrada con otro usuario
/aceptar	/aceptar ¡id _i ! — Acepta la solicitud y completa el handshake ECDH
/enviar	/enviar ¡mensaje _i ! — Cifra con Grover+AES y entrega al destinatario
/descifrar	/descifrar ¡base64 _i ! — Descifra manualmente un texto recibido
/estado	Muestra estado de sesión: pareja, clave derivada, mensajes enviados

2) *Flujo completo del protocolo en Telegram*: El siguiente es el flujo paso a paso de una sesión cifrada entre dos usuarios de Telegram:

- 1) Ambos usuarios abren el bot y ejecutan /miid para obtener su Telegram User ID numérico.
- 2) Ambos ejecutan /nuevasesion. El bot crea internamente un objeto SesiónCifradaV2 para cada uno y genera un par de claves X25519. La clave privada nunca sale del servidor.
- 3) El usuario A ejecuta /conectar ¡id_B!. El bot registra la solicitud en el diccionario solicitudes_pendientes y envía una notificación automática al usuario B.
- 4) El usuario B ejecuta /aceptar ¡id_A!. El bot llama a SessionManager.completar_handshake() que ejecuta el intercambio ECDH: cada sesión usa la clave pública del otro para calcular el secreto compartido, luego kdf_propio() deriva la clave simétrica.

- 5) Ambos usuarios reciben un mensaje de confirmación que incluye los primeros bytes de la clave derivada (idéntica en ambos lados, verificable visualmente).
- 6) Desde este punto, cualquier texto que escriba el usuario A (con /enviar o directamente) pasa por el pipeline cifrar() antes de transmitirse.
- 7) El bot entrega el mensaje al usuario B y simultáneamente llama a descifrar() con la sesión de B para mostrarle el texto original junto con el Base64 que viajó por Telegram.

D. Chat cifrado cliente/servidor WiFi

1) *Arquitectura de red:* El sistema cliente/servidor WiFi demuestra el protocolo sobre TCP/IP real sin depender de ningún servicio de terceros. Los mensajes viajan por la red local como texto cifrado, y cualquier herramienta de análisis de red como Wireshark solo vería el Base64 ilegible.

- **servidor.py:** corre en la laptop. Escucha en todas las interfaces de red (0.0.0.0) en el puerto 8888. Al iniciar detecta automáticamente la IP local WiFi y la muestra. Acepta múltiples clientes con un hilo por conexión.
- **cliente.py:** corre en cualquier otro dispositivo de la red. Recibe la IP del servidor como argumento en la línea de comandos. Establece una conexión TCP y ejecuta el handshake ECDH automáticamente.

2) *Protocolo de mensajes JSON:* Los dos procesos se comunican con JSON delimitado por saltos de línea. Cada mensaje tiene un campo tipo que indica su propósito:

```
{ "tipo": "handshake_pub",
  "pub_key": "BASE64_32_bytes..." }
{ "tipo": "handshake_ok",
  "msg": "Sesion cifrada establecida" }
{ "tipo": "mensaje",
  "cifrado": "BASE64_cifrado..." }
{ "tipo": "fin" }
```

3) *Flujo del handshake en red real:* El establecimiento de una sesión segura en GrovLock se realiza mediante un proceso de intercambio de claves basado en X25519 sobre una conexión TCP, y este procedimiento permite que ambas partes generen un secreto compartido para la derivación de las claves utilizadas durante la comunicación cifrada.

- 1) El cliente abre una conexión TCP al servidor (socket.connect)
- 2) El cliente envía su clave pública X25519 en formato JSON
- 3) El servidor responde con su propia clave pública X25519
- 4) Ambos llaman a completar_handshake() con la clave recibida usando el mismo sesion_id=1337
- 5) El servidor envía handshake_ok como confirmación de que la sesión está lista
- 6) Desde este punto cada mensaje pasa por cifrar() antes de enviarse y descifrar() al recibirlo
- 7) Un hilo independiente (recibir_loop) escucha mensajes entrantes sin bloquear la entrada del usuario.

E. Evaluación

El primer entorno consistió en una arquitectura cliente-servidor basada en sockets TCP, donde el cliente establece una conexión con el servidor, ejecuta el intercambio de claves mediante X25519, deriva la clave de sesión correspondiente, aplica la capa de difusión GrovLock sobre los datos y finalmente cifra la información utilizando AES-256-GCM antes de su transmisión. Una vez recibido el mensaje, el servidor verifica la autenticidad de los datos, ejecuta el proceso de descifrado y aplica la transformación inversa de la capa de difusión para recuperar el mensaje original.

Como segundo entorno experimental, se desarrolló un bot de Telegram que integra completamente el esquema GrovLock dentro del flujo de comunicación de la plataforma. El sistema permite que dos usuarios establezcan una sesión cifrada mediante un proceso de intercambio de claves basado en X25519, generando una clave de sesión compartida a partir de la cual se derivan los parámetros criptográficos utilizados por el algoritmo. Una vez completado el proceso de establecimiento de sesión, los mensajes son sometidos a la capa de difusión inspirada en el operador de Grover y posteriormente cifrados mediante AES-256-GCM antes de ser enviados a través de la infraestructura de Telegram. Como resultado, la plataforma únicamente transporta cadenas cifradas codificadas en Base64, mientras que el contenido original permanece protegido mediante el esquema criptográfico propuesto. En el extremo receptor, el mensaje es autenticado, descifrado y reconstruido automáticamente utilizando la misma clave de sesión compartida.

La implementación desarrollada incorpora además funcionalidades para la generación dinámica de sesiones, establecimiento de conexiones entre usuarios, intercambio seguro de claves, cifrado y descifrado automático de mensajes, permitiendo evaluar el comportamiento de GrovLock en un entorno de mensajería instantánea ampliamente utilizado.

III. RESULTADOS

Con el objetivo de validar el funcionamiento del algoritmo propuesto en un entorno de comunicación real, se desarrolló una aplicación telemática basada en la plataforma Telegram; la elección de Telegram se debe a que proporciona una infraestructura de mensajería ampliamente utilizada, permitiendo evaluar el comportamiento del esquema criptográfico en condiciones reales de uso y sobre redes públicas de Internet.

Es importante destacar que Telegram ya implementa mecanismos propios de protección de datos mediante el protocolo MTPProto; sin embargo, en este trabajo se añadió una capa criptográfica adicional que opera independientemente de la seguridad proporcionada por la plataforma. De esta manera, los mensajes son cifrados por el algoritmo HybridECDH-Grover-AES antes de ser enviados a Telegram, logrando un esquema de cifrado extremo a extremo a nivel de aplicación.

Como resultado, los servidores de Telegram únicamente transportan cadenas codificadas en Base64 que contienen información previamente cifrada, lo que significa que el contenido original del mensaje solamente puede ser recuperado

por los usuarios que hayan completado correctamente el proceso de intercambio de claves y compartan la misma clave de sesión derivada.

A. Componentes principales del sistema

La arquitectura desarrollada se compone de tres módulos principales.

1) *Módulo de comunicación (bot.py)*: Este componente constituye el punto de interacción entre los usuarios y el sistema. Se encarga de registrar los comandos disponibles, recibir solicitudes, coordinar el intercambio de mensajes y comunicarse con la API oficial de Telegram. Asimismo, actúa como intermediario entre los usuarios y el módulo criptográfico.

```
474 def main():
475     print("=" * 60)
476     print(" HybridECDH-Grover-AES Bot - Iniciando...")
477     print("=" * 60)
478
479     app = Application.builder().token(TOKEN).build()
480
481     # Registrar handlers
482     app.add_handler(CommandHandler("start", cmd_start))
483     app.add_handler(CommandHandler("miid", cmd_miid))
484     app.add_handler(CommandHandler("nuevasesion", cmd_nueva_sesion))
485     app.add_handler(CommandHandler("conectar", cmd_conectar))
486     app.add_handler(CommandHandler("aceptar", cmd_aceptar))
487     app.add_handler(CommandHandler("enviar", cmd_enviar))
488     app.add_handler(CommandHandler("descifrar", cmd_descifrar))
489     app.add_handler(CommandHandler("estado", cmd_estado))
490     app.add_handler(CommandHandler("ayuda", cmd_ayuda))
491     app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, mensaje_libre))
492
493     print(" Bot corriendo. Presiona Ctrl+C para detener.")
494     print("=" * 60)
495     app.run_polling(allowed_updates=Update.ALL_TYPES)
496
497
498 if __name__ == "__main__":
499     main()
```

Fig. 8. Código para el módulo de comunicación

2) *Módulo de gestión de sesiones (session_manager.py)*: Este módulo administra toda la información relacionada con las sesiones activas y entre sus responsabilidades se encuentran la generación de sesiones, el almacenamiento temporal de solicitudes de conexión, la asociación entre usuarios conectados y la gestión de las claves derivadas obtenidas durante el proceso ECDH; además, mantiene los contadores de mensajes necesarios para garantizar la unicidad de los nonces utilizados por AES-GCM.

```
16 class EstadoSesion:
17     """Estado de la sesión entre dos usuarios."""
18     ESPERANDO_HANDSHAKE = "esperando_handshake"
19     ACTIVA = "activa"
20
21     def __init__(self, owner_id: int):
22         self.owner_id = owner_id
23         self.sesion = SesionCifradoV2(str(owner_id))
24         self.estado = self.ESPERANDO_HANDSHAKE
25         self.partner_id: Optional[int] = None
26
27     @property
28     def lista(self) -> bool:
29         return self.estado == self.ACTIVA
```

Fig. 9. Código clave para el módulo de gestión de sesiones

3) *Núcleo criptográfico (hybrid_cipher_v2.py y grover_layer.py)*: Este componente implementa la totalidad del algoritmo propuesto, así, incluye la generación de pares de claves X25519, la derivación de claves mediante SHA-256, la capa de difusión inspirada en Grover y el

cifrado autenticado mediante AES-256-GCM; también, cada vez que un usuario envía o recibe un mensaje, este módulo ejecuta automáticamente las operaciones criptográficas necesarias para garantizar la confidencialidad e integridad de la información.

```
1
2
3 CAPA DE GROVER — Difusor cuántico simulado
4
5 Operador de difusión: D(v)[i] = (2μ - v[i]) mod 256
6 donde μ = mean(v) — basado en Grover (1996)
7
```

Fig. 10. Funcionalidad de la capa de grover

```
1
2
3 HybridECDH-Grover-AES — Versión 2.0
4
5 Fase 1 — ECDH X25519 : intercambio de claves
6 Fase 2 — KDF personalizada : derivación SHA-256 cascada
7 Fase 3 — Difusor de Grover : difusión cuántica simulada
8 Fase 4 — AES-256-GCM : cifrado autenticado
9
10
```

Fig. 11. Fases de HybridECDH-Grover-AES

B. Flujo completo del protocolo en Telegram

El proceso de comunicación segura desarrollado puede dividirse en varias etapas consecutivas.

1) *Identificación de usuarios*: Inicialmente, ambos participantes obtienen su identificador único de Telegram mediante el comando /miid. Este identificador permite asociar posteriormente las sesiones criptográficas con usuarios específicos dentro del sistema.

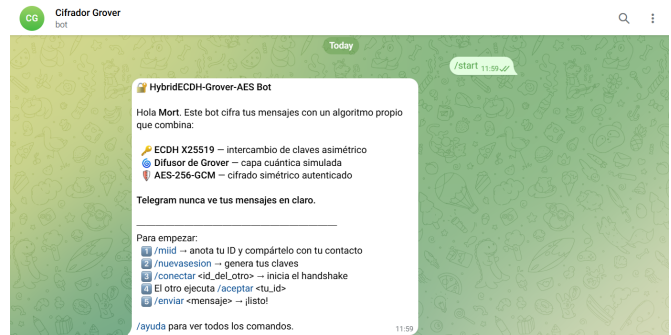


Fig. 12. Inicio de conversación en Telegram

2) *Generación de claves*: Posteriormente cada usuario ejecuta el comando /nuevasesion. En esta etapa el sistema crea internamente un objeto de tipo SesionCifradoV2, el cual genera automáticamente un par de claves criptográficas X25519 compuesto por una clave privada y una clave pública. La clave privada permanece almacenada exclusivamente dentro de la sesión correspondiente y nunca es transmitida ni mostrada al usuario.

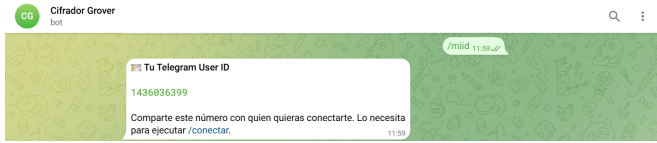


Fig. 13. Generación de un User ID en Telegram

3) *Solicitud de conexión:* Una vez generadas las claves, el usuario que desea iniciar la comunicación ejecuta el comando: `/conectar [id_destino]`, de esta manera, el sistema registra la solicitud y notifica automáticamente al usuario destino que existe una petición pendiente de conexión cifrada.



Fig. 14. Mensaje con clave pública en Telegram

4) *Intercambio ECDH y derivación de clave:* Cuando el segundo usuario acepta la solicitud mediante: `/aceptar [id_origen]` el sistema ejecuta automáticamente el protocolo ECDH, durante esta etapa ambas sesiones intercambian únicamente sus claves públicas y calculan localmente el mismo secreto compartido utilizando sus respectivas claves privadas. Posteriormente dicho secreto es procesado por la función de derivación de claves basada en SHA-256 para producir una clave simétrica de 256 bits; entonces, esta clave será utilizada durante toda la sesión para cifrar y descifrar mensajes.

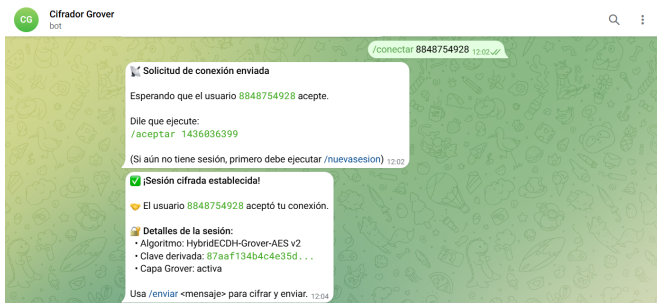


Fig. 15. Conexión con el User ID

5) *Transmisión de mensajes cifrados:* Una vez completado el intercambio de claves, los usuarios pueden comenzar a intercambiar mensajes. Cada mensaje enviado atraviesa las siguientes etapas:

6) *Intercambio de mensajes:* Posteriormente se enviaron múltiples mensajes de prueba en ambas direcciones, se observó que Telegram únicamente transportaba cadenas codificadas en Base64 generadas por el proceso de cifrado, mientras que el contenido original era recuperado correctamente únicamente por el destinatario autorizado. Cabe resaltar que durante las

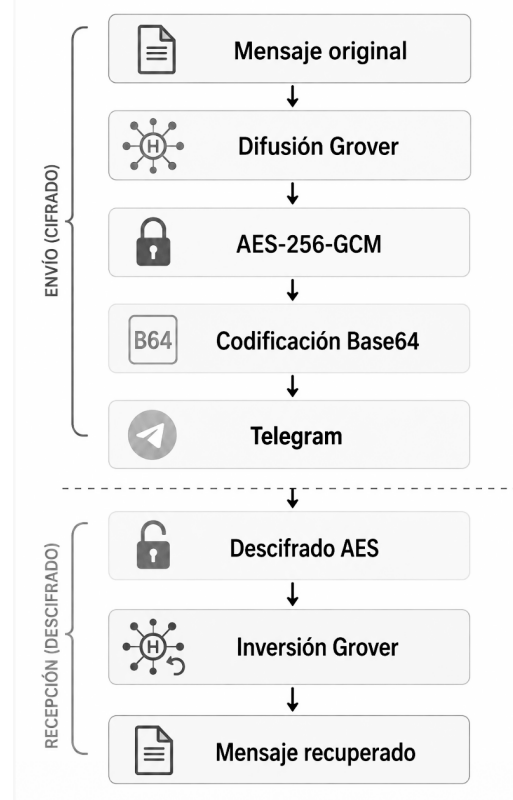


Fig. 16. Flujo de las etapas

pruebas no se detectaron pérdidas de información ni inconsistencias en el proceso de cifrado y descifrado.

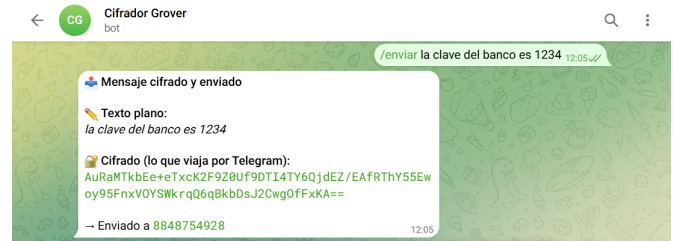


Fig. 17. Mensaje cifrado enviado

IV. DISCUSIÓN

Los resultados obtenidos demuestran que GrovLock puede implementarse exitosamente en entornos reales de comunicación telemática mediante la integración de intercambio de claves X25519, derivación de claves basada en SHA-256, una capa de difusión inspirada en el operador de Grover y cifrado autenticado AES-256-GCM; sobre todo en la aplicación de mensajería desarrollada sobre Telegram, el algoritmo permitió establecer sesiones seguras, intercambiar claves de manera transparente y recuperar correctamente los mensajes originales después del proceso de descifrado.

La incorporación de una capa de difusión previa al cifrado constituye el principal aporte de la propuesta. A diferencia



Fig. 18. Mensaje descifrado desde otro móvil (celular)

de los esquemas híbridos convencionales, que generalmente combinan únicamente mecanismos de intercambio de claves y cifrado autenticado, GrovLock introduce una transformación reversible inspirada en el operador de difusión de Grover y esta etapa incrementa la dispersión de los datos antes de la aplicación de AES-256-GCM, reduciendo la presencia de patrones reconocibles en el texto plano y añadiendo una capa adicional de transformación. Aunque esta operación no reemplaza las garantías criptográficas proporcionadas por AES, puede contribuir a aumentar la complejidad estructural de los datos procesados.

Los resultados también evidencian que el uso de X25519 permitió generar secretos compartidos de manera eficiente y segura sin transmitir información sensible a través de la red. Este comportamiento coincide con lo reportado por Nguyen et al. [6], quienes destacan la eficiencia y seguridad de Curve25519 en protocolos modernos de intercambio de claves. Asimismo, la utilización de AES-256-GCM permitió garantizar simultáneamente confidencialidad, integridad y au-

tenticidad de los mensajes transmitidos, concordando con los hallazgos reportados por Mandal et al. [3] sobre la robustez de los esquemas AEAD en aplicaciones de comunicación segura.

Por su lado, la implementación en Telegram permitió comprobar la viabilidad práctica del algoritmo en plataformas de mensajería ampliamente utilizadas, así, los mensajes transmitidos fueron observados únicamente como cadenas cifradas codificadas en Base64, mientras que el contenido original permaneció accesible únicamente para los usuarios que compartían la misma clave de sesión. Este comportamiento demuestra que GrovLock puede funcionar como una capa criptográfica complementaria independiente de los mecanismos de seguridad ya proporcionados por la plataforma.

Como trabajo futuro, se propone realizar evaluaciones de rendimiento comparativas frente a esquemas híbridos tradicionales, incorporar métricas de análisis estadístico sobre los datos transformados y desarrollar estudios criptográficos más profundos que permitan determinar el impacto real de la capa de difusión inspirada en Grover sobre la seguridad del sistema. Además, sería interesante explorar la adaptación de la propuesta a entornos de comunicación de gran escala, aplicaciones IoT y arquitecturas distribuidas donde la protección de la información constituye un requisito crítico.

V. CONCLUSIONES

El presente trabajo permitió diseñar e implementar GrovLock, un esquema criptográfico híbrido orientado a la protección de comunicaciones telemáticas; la integración de X25519 para el intercambio de claves, una función de derivación basada en SHA-256, una capa de difusión inspirada en Grover y AES-256-GCM permitió establecer sesiones seguras y proteger eficazmente la información intercambiada entre usuarios. Es así como, la incorporación de una capa de difusión reversible inspirada en el operador de Grover constituye el principal aporte de la propuesta. Esta transformación incrementa la dispersión de los datos antes del proceso de cifrado, añadiendo una etapa adicional de mezcla sin afectar la capacidad de recuperación del mensaje original ni la compatibilidad con algoritmos criptográficos ampliamente utilizados.

La implementación del algoritmo en un bot de Telegram permitió validar su aplicabilidad en un entorno real de mensajería instantánea. Los resultados evidenciaron que los mensajes enviados a través de la plataforma eran transportados únicamente como texto cifrado, mientras que el contenido original permanecía accesible exclusivamente para los usuarios autorizados que compartían la misma clave de sesión. Es por eso que, el uso de AES-256-GCM como mecanismo de cifrado autenticado permitió proporcionar simultáneamente confidencialidad, autenticación e integridad de los datos transmitidos. De esta forma, la combinación de este algoritmo con la etapa de difusión propuesta y el intercambio seguro de claves generó una arquitectura multicapa capaz de fortalecer la protección de las comunicaciones telemáticas frente a diversas amenazas.

En síntesis, el desarrollo de GrovLock permitió demostrar la viabilidad de integrar diferentes mecanismos criptográficos

en una arquitectura híbrida orientada a la protección de comunicaciones telemáticas; la combinación de intercambio seguro de claves, derivación criptográfica, difusión de datos y cifrado autenticado posibilitó la construcción de un sistema funcional capaz de garantizar la confidencialidad, integridad y autenticidad de la información transmitida. Del mismo modo, la implementación y validación en entornos reales evidenciaron el potencial de este tipo de propuestas para fortalecer la seguridad de las comunicaciones digitales y servir como base para futuras investigaciones en el ámbito de la criptografía aplicada.

REFERENCES

- [1] Nhu-Quynh Luc, Tat-Thang Nguyen, and Thanh Toan Dao, "Developing Secure Messaging Software Using Post-Quantum Cryptography," *Engineering, Technology & Applied Science Research*, vol. 13, no. 6, pp. 12440–12445, 2023, doi: 10.48084/etasr.6549.
- [2] Fauziyah, Z. Wang, and M. Tabassum, "A Holistic Secure Communication Mechanism Using a Multilayered Cryptographic Protocol to Enhanced Security," *Computers, Materials & Continua*, vol. 78, no. 3, 2024, doi: 10.32604/cmc.2024.046797.
- [3] Surajit Mandal, Ravi Anand, Mostafizar Rahman, Santanu Sarkar, and Takanori Isobe, "Implementing Grover's on AES-Based AEAD Schemes," *Sci. Rep.*, vol. 14, 2024, doi: 10.1038/s41598-024-69188-8.
- [4] Chun Seok Yoon, Chang Ho Hong, Min Sung Kang, Ji-Woong Choi, and Hyung Jin Yang, "Quantum Asymmetric Key Crypto Scheme Using Grover Iteration," *Sci. Rep.*, vol. 13, 2023, doi: 10.1038/s41598-023-30860-0.
- [5] A. Al-Bayat and M. Perkowski, "A Concept of Controlling Grover Diffusion Operator: A New Approach to Solve Arbitrary Boolean-Based Problems," *Sci. Rep.*, vol. 14, 2024, doi: 10.1038/s41598-024-74587-y.
- [6] H. Nguyen, T. Hoang, and L. Tran, "Efficient Hardware Implementation of Elliptic-Curve Diffie–Hellman Ephemeral on Curve25519," *Electronics (Basel)*, vol. 12, no. 21, p. 4480, 2023, doi: 10.3390/electronics12214480.
- [7] C. W. Chuah, N. Z. Harun, and I. R. A. Hamid, "Key Derivation Function: Key-Hash Based Computational Extractor and Stream Based Pseudorandom Expander," *PeerJ Comput. Sci.*, vol. 10, p. e2249, 2024, doi: 10.7717/peerj-cs.2249.
- [8] A. S. Bhati, A. Dufka, E. Andreeva, A. Roy, and B. Preneel, "Skye: An Expanding PRF based Fast KDF and its Applications," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (AsiaCCS '24)*, 2024, doi: 10.1145/3634737.3637673.